

Бинарное дерево - это дерево, каждый узел которого имеет максимум два потомка.
Для любой вершины выполняется **свойство бинарного дерева поиска**:

Если x - узел бинарного дерева поиска, а узел y находится в левом поддереве x , то $y.key \leq x.key$. Если узел y находится в правом поддереве x , то $x.key \leq y.key$.

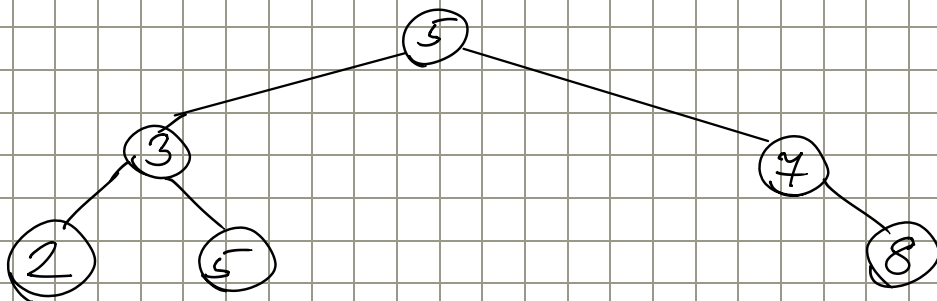
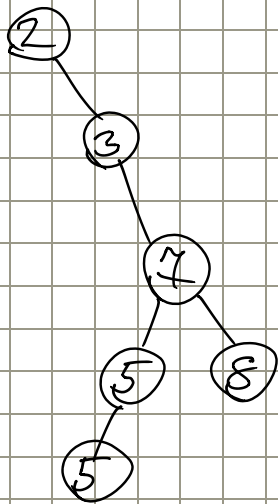


рис. 5.1



На рисунках 5.1, 5.2 показано выполнение этого св-ва, но дерево на 5.1 эффективнее, т.к. его высота равна 2, а 5.2. высота 4.

рис. 5.2

Для того, чтобы вывести листы дерева есть несколько алгоритмов его обхода:

- 1) **центрированный (inorder) обход дерева**:
ключ в корне поддерева выводится между значениями

левого и правого поддерева;

2) в прямой (preorder) порядке: сначала выводится корень, а потом значения левого и правого поддерева;

3) в обратном (postorder) порядке: сначала значения левого и правого потом корня.

Поиск:

Для поиска узла выполняется процедура

Tree Search, которая получает указатель на корень дерева и ключ k , а возвращает указатель на найденную вершину, либо null.

Tree Search(x, k):

```
1 if  $x = \text{null}$  или  $k = x.\text{key}$ :  
2   return  $x$   
3 if  $k < x.\text{key}$ :  
4   return TreeSearch( $x.\text{left}, k$ )  
5 else:  
6   return TreeSearch( $x.\text{right}, k$ )
```

В худшем случае должно выполняться кол-во операций сравнения равное высоте дерева. След, сложность $O(h)$, где h - высота дерева.

Вставка:

Для вставки нового значения в дерево воспользуемся `Tree Insert`. В качестве параметров передается указатель на вершину `T` и значение `n`, которое хотим вставить.

`Tree Insert(T, n):`

```
1 if T = null:
2     T = new Node()
3     T.left = null
4     T.right = null
5     T.key = n
6     return
7 if n < T.key:
8     TreeInsert(T.left, n)
9 else if n > T.key:
10    TreeInsert(T.right, n)
```

Процедура также выполняется за $O(h)$.

Удаление:

Для удаления потребуется процедура `Tree Delete`, которой передается указатель на корень и значение, которое нужно удалить `n`.

В процедуре рассматриваются три возможных варианта удаления:

1) у вершины нет детей, то просто удаляем;

2) у вершины один потомок, в таком случае мы удалим вершину, а ребенок встанет на его место;

3) у вершины два потомка, тогда ищем в правой поддереве минимальный или в левой максимальный элемент, заменяем значение ключа у вершины и удаляем дубинку, то есть мы. мы делаем элемент.

Tree Delete(T, n):

```
1 if  $T = \text{null}$ :  
2     return  
3 if  $n < T.\text{key}$ :  
4     Tree Delete( $T.\text{left}, n$ )  
5 if  $n > T.\text{key}$ :  
6     Tree Delete( $T.\text{right}, n$ )  
7  $\text{temp} = T$   
8 if  $T.\text{left} = \text{null}$  и  $T.\text{right} = \text{null}$ :  
9     delete  $T$   
10     $T = \text{null}$   
11    return;  
12 if  $T.\text{left} = \text{null}$ :  
13     $T = T.\text{right}$   
14    delete  $\text{temp}$   
15    return;  
16 if  $T.\text{right} = \text{null}$ :  
17     $T = T.\text{left}$ 
```

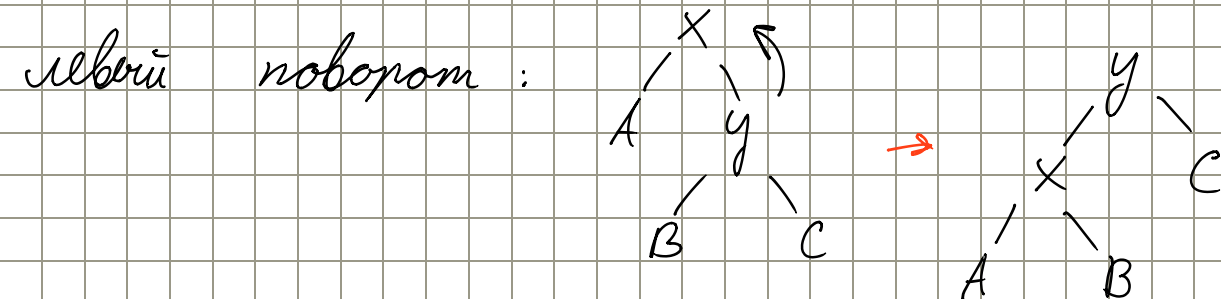
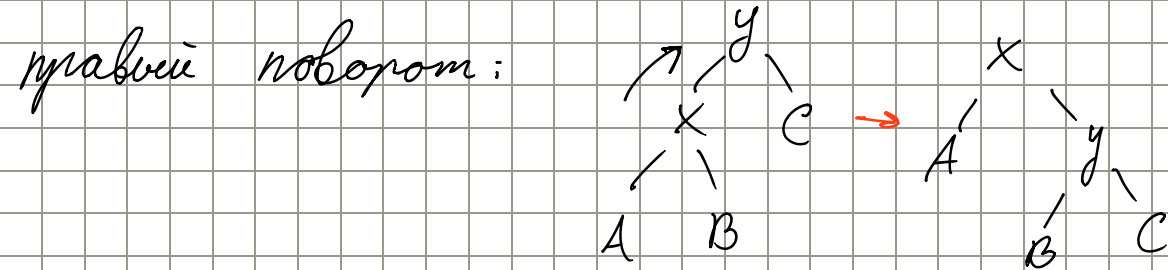
```

18 delete temp
19 return;
20 minNode = FindMin(T.right)
21 T.key = minNode.key
22 TreeDelete(T.right, minNode.key)

```

Процедура выполн. за $O(h)$.

Повороты:
используются для балансировки дерева (AVL, KЧ).



Работает за $O(1)$.